Shortest Path in Binary Matrix (medium)

Problem Statement

Given a matrix of size `m x n`, return the `shortest path` from the top-left corner `(0,0)` to the bottom-right corner `(N-1, N-1)`. If there is no such path, the output should be `-1`.

Follow the rules below while calculating the shortest path.

- You can move horizontally, vertically, and diagonally to adjacent cells.
- The visited cell should be `0`.
- The path length is counted as the number of steps taken.

Examples

1. **Example 1**:

- **Input**:

```
[[0, 1, 0],
 [0, 0, 0],
 [0, 1, 0]]
```

- **Expected Output**: `3`
- **Justification**: The shortest path is (0, 0) -> (1, 1) -> (2, 2), totaling 3 steps.

2. **Example 2**:

- **Input**:

```
[1, 1, 1],
 [0, 1, 1],
 [0, 0, 0]]
```

- **Expected Output**: `-1`
- **Justification**: The top-left cell can't be visited, as its value is 1. So, it returns -1.

3. **Example 3**:

- **Input**:

```
[[0, 0, 0],
 [0, 1, 0],
 [0, 0, 0]]
```

- **Expected Output**: `4`
- **Justification**: The shortest path is (0, 0) -> (1, 0) -> (2, 1) -> (2, 2), totaling 4 steps.

Try it yourself

Try solving this question here:

Java

```java
import java.util.LinkedList;
import java.util.Queue;

public class Solution {
    public int shortestPathBinaryMatrix(int[][] grid) {
        // ToDo: Write Your Code Here.
        return -1;  // No path found.
    }
}
```

Test Case | Run Code Result | Stdout

[[0,1,0],[0,0,0],[0,1,0]]

Use Example Testcases | Run | Submit

← Previous
Solution: Diagonal Traverse

Next →
Solution: Shortest Path In Binary Matrix

Mark as Completed